6

PART 2: FRAMEWORK

Architecture as Context

The Riverside Clinics booking specification from the last chapter is finished, and three developers pick it up the same afternoon. One takes patient lookup. One takes slot availability. One takes doctor eligibility. Each opens an AI agent, describes a slice of the use case, and gets back working code in minutes: methods that compile, pass their own tests, and do exactly what was asked. All three branches are ready to merge into the same feature by end of day.

Inconsistent Implementations

Open the three branches side by side and each one is defensible on its own. The patient-lookup developer's agent produced a `PatientLookupService` that calls a repository, which calls the database, which is exactly how the rest of Riverside's system is built. The slot-availability developer's agent produced a `DoctorAvailabilityManager` that queries the database directly from what should have been the application layer, skipping the repository entirely because no rule in the project said a repository belonged there. The doctor-eligibility developer's agent, asked to check whether a doctor could take a given slot, decided the cleanest design was to publish an internal event once eligibility was confirmed, an event no other part of the system subscribes to, or ever will.

Each of the three passed its own unit tests. None of the three is wrong in isolation. What they share is that none of the three agents was told what kind of object this is supposed to be, so each one reached for a different, individually reasonable answer: a manager here, a service there, an event publisher somewhere a plain return value would have done the job. A reviewer looking at the merged branch is looking at three conventions where the project needs one, and a layer boundary one of the three breaks through.

This is the cost that doesn't show up until the second or third feature. The first divergence looks like a style nitpick, worth a comment and a quick fix. By the tenth feature, with no rule ever written down, the project's real architecture is whichever pattern happened to get written most often, and a new engineer reading the code has no way to tell a decision from an accident. Every mature codebase already runs on rules like these: controllers never call a repository directly, only the domain layer is allowed to raise a business event, validation happens in the application layer and nowhere else. Developers absorb rules like that by reading months of other people's pull requests. An AI agent sees only the request in front of it. It has no pull requests to have read.

Architecture in This Book

Call this what it is: **architecture**. In the sense this book uses the word, architecture is the set of decisions that apply to every feature a team builds rather than just the one in front of it, and that change far more slowly than requirements do: which layer is allowed to call which, what a repository gets named, where a business rule is allowed to live, how a module's boundary gets drawn. A diagram unsupported by rules like these is illustration. A single sentence that states one of these rules, with no box drawn at all, is architecture.

Every one of Riverside's three implementations obeyed a perfectly valid, textbook pattern. None of the agents had been told which pattern this project uses, because the pattern wasn't written down anywhere an AI agent could read it. That gap runs under everything in this chapter, and points at a change bigger than one codebase. For a few years, getting good output from an AI agent meant refining the prompt: finding the right wording, trying it, retrying it, until the result looked acceptable. Architecture reframes that problem. Instead of wording each request more carefully, a team writes its structural rules once, in a document, and every future request gets shorter, because the rules no longer need repeating.

That change has a name: **context**, the written information an AI agent is given about a project before it is asked to do anything, and context engineering, the discipline of building that information up once so it doesn't need re-explaining request by request. Architecture is the first piece of context in this book that scales across an entire codebase instead of one feature at a time.

> **Architecture settles the structural questions before AI ever reaches the business problem: which layer, which name, which boundary. What's left is the business decision itself, and the rest of the specification has already answered that one.**

Documenting Existing Rules

An architecture that lives only in habit is still an architecture. The work here is finding the rules already in force and putting them somewhere an AI agent can read them before it starts guessing.

1. Name the layers your system actually has, and give each one exactly one responsibility. Three or four layers is typical for a system Riverside's size; more than that usually means two layers are doing the same job under different names.
2. State the dependency rule for those layers in plain sentences: which layer may depend on which, and which layer, usually the one holding business rules, may depend on nothing else in the system at all.
3. Fix one naming convention per kind of artifact: application service, repository, event, command. Write it down once, in one place every developer's agent can read.
4. Record the cross-cutting decisions that don't fit neatly into layers or names: where errors surface, where security is enforced, what gets tested at which layer, deployment. Use a documentation format that already exists: an arc42 template, a widely used structure for architecture documentation; architecture decision records (ADRs), short files that each capture one decision and the reason behind it; or C4 diagrams, a set of architecture views at increasing levels of zoom, from the whole system down to individual components. Simon Martinelli, whose specification-driven development methodology this book draws on throughout, recommends arc42. The format is widely known and already built to hold exactly this content. What matters is that the decisions end up somewhere durable.

Worked Example: Riverside Clinics Architecture Rules

Riverside has been building software this way for years without ever writing the rules down. Here is what the team produces the week after the three-branch merge above, the first time anyone puts the project's actual layering and naming in writing.

Layers and the Dependency Rule

Presentation: handles requests from patients, staff, and the clinic's booking website
Application: coordinates one use case at a time, such as Schedule Appointment
Domain: the business rules for patients, doctors, appointments, and slots
Infrastructure: persistence, the scheduling database, outbound notifications

- `Presentation may depend on Application. Nothing may depend on Presentation.`
- `Application may depend on Domain. Nothing outside Application may call Infrastructure directly.`
- `Infrastructure may depend on Domain, to read and store domain objects.`
- `Domain depends on nothing else in the system. It is the one layer every other layer is written around.`

Worked Example, Continued: Naming Conventions

Naming, Fixed Once per Kind of Object

- An application service ends in Service: PatientService, not PatientManager, not PatientProcessor.
- A repository ends in Repository: DoctorAvailabilityRepository.
- A domain event ends in Event: AppointmentScheduledEvent, raised only from the domain layer.
- A use-case command is named for the action it performs: BookAppointmentCommand, CancelAppointmentCommand.

Run the three-developer scenario again with this document sitting in the AI agent's context, and the result changes shape. Patient lookup, slot availability, and doctor eligibility all become one kind of object, an application service depending on the domain, backed by a repository in infrastructure. Eligibility becomes a plain answer from a domain rule instead of an event with no subscribers. Three names for the same idea become one, and the layer boundary that broke on the first pass has nowhere left to break through.

Limitations

▲ **WARNING**

A document is not enforcement. Rules on a page decay the same way tribal knowledge did, unless someone checks the code against them; a repository that starts calling another repository directly will still pass every test that isn't looking for it. Pair the document with something automated, a dependency-direction linter or an architecture test, that fails a build the moment the written rule and the actual code stop agreeing.

Writing rules too rigid for the system's actual size	A four-person team adopts a layering scheme built for an organization many times its size because a template recommended it, and every trivial change now requires touching five files to satisfy a rule the project never needed.
Treating architecture as a substitute for the domain model	The layers are correct, the naming is consistent, and the business rule inside the domain layer is still wrong, because architecture governs where a decision lives, not whether the decision itself was made correctly.

Template: Architecture Document

- ☐ Layers named, each with exactly one stated responsibility.
- ☐ The dependency rule written as plain sentences: which layer may call which, and which layer depends on nothing.
- ☐ A naming convention fixed per kind of artifact: service, repository, event, command.
- ☐ A module-boundary rule: organized by business capability, patients, appointments, billing, rather than by technical role.
- ☐ Which layer owns error handling, logging, and security enforcement, with the format each one takes left to a skill (Chapter 7).
- ☐ Which layer needs a unit test and which needs an integration test; the framework and test naming are a skill's business.
- ☐ A short decision log, one entry per architectural choice and its reason, for anything that doesn't fit cleanly into the sections above.

This is deliberately smaller than a full arc42 template or a complete C4 diagram set. Start with the sections above; grow into arc42, ADRs, or C4 as the team and the decisions worth recording both grow.

Summary

- Code that is entirely correct can still be wrong for a project, if the AI agent doesn't know which structural rules this project follows.
- Every mature codebase already runs on invisible rules; developers absorb them by reading each other's work over months, and an AI agent has no equivalent history to learn from.
- Architecture is the set of decisions that apply to every feature a team builds: layering, dependency direction, naming, module boundaries. A diagram only illustrates those decisions; it isn't one of them.
- The dependency rule is the one sentence that does the most work: once the independent layer is named, most accidental coupling stops before it starts.
- Writing architecture down once is what turns prompt engineering, wording each request carefully, into context engineering, giving the AI the rules once so no future request has to restate them.

DISCUSSION QUESTIONS

- If three developers each asked an AI agent to add a service class this week, would the three results share one naming convention, or three?
- Where does your team's dependency rule currently live, and who would a new hire ask to find it?

PRACTICAL EXERCISE

Pick one system you maintain and write down its layers and its dependency rule in five sentences or fewer. If you can't finish in five sentences, that's the rule your next AI-generated pull request is about to violate.

Architecture as Context ends here.